

Part 3: Engineering Plan & Proof of Concept

Part 1 delivers a working single-URL crawler deployed on GCP Cloud Run. Part 3 describes how to scale it into the distributed system designed in Part 2.

Implementation Plan

Phase 1 — Core Crawl Pipeline (1 Week)

Build: - URL ingestion service (read from GCS file or Cloud SQL, normalize, deduplicate via Redis Bloom filter, publish to Pub/Sub) - Crawler workers on GKE subscribing to Pub/Sub crawl topic - Shared per-domain rate limiting via Redis so all worker pods see the same counter per domain (Part 1 rate limiting is per-instance in memory — doesn't work when 100 pods are crawling the same domain independently) - Raw HTML upload to GCS on successful crawl

Deliverable: Distributed crawling functional — URLs flow from input file through the queue into GKE workers, raw HTML lands in GCS.

Phase 2 — Extraction Pipeline (1 Week)

Build: - Redis write buffer + DB flusher for Cloud SQL writes - Content hash comparison to skip unchanged pages - Extraction workers on GKE subscribing to extraction topic - Batch KeyBERT topic extraction - Bulk write to Elasticsearch

Deliverable: End-to-end pipeline functional — crawled pages are parsed, classified, topics extracted, and searchable in Elasticsearch.

Phase 3 — Reliability & Observability (1 Week)

Build: - Dead Letter Topic after 5 retries with structured error metadata - Cloud Monitoring dashboards (queue backlog, throughput, error rate, P95 latency) - Alerting

on all SLO thresholds defined in Part 2 - Stats table queries surfaced as internal dashboards for per-domain visibility

Deliverable: Production-operable pipeline — failures are captured, metrics are visible, alerts fire on degradation.

Phase 4 — Scale Validation & Load Testing (1 Week)

Load test at 10% scale (100M URLs), then full 1B: - Measure actual crawl and extraction latency under sustained GKE load — the 1s/3s estimates from Part 2 are based on local testing and need to be validated - Sustained throughput at target URLs/sec for 1+ hour - Queue lag under load (Pub/Sub oldest unacked message age) - CPU and memory utilisation per instance type — validate or revise instance count estimates from Part 2 - Failure handling end-to-end: confirm Dead Letter Topic captures failures, confirm retry recovers correctly

Deliverable: Load test report with actual throughput, latency, and resource utilisation figures. Revised instance count estimates if actual numbers differ from Part 2. Go/no-go recommendation for production.

Potential Blockers

Things to monitor during Phase 4 - Playwright memory behaviour — Chromium's memory usage under hours of sustained concurrent crawling is unknown. During load testing we need to track per-pod memory over time. If leaks are observed, a browser restart policy (recycle after N pages or M minutes) would be the fix — but we don't know if this will be needed until we run it. - **Actual latency numbers** — the 1s crawl / 3s extraction estimates come from local single-instance testing. Real behaviour under sustained GKE load with real domain traffic needs to be measured in Phase 4 before instance count projections from Part 2 can be trusted.

Genuine unknowns - IP reputation blocking — whether GKE egress IPs are blocked by bot-protection systems on target domains won't be known until the crawl runs. Residential proxy integration is the reliable fix but adds cost and vendor dependency — this is a product decision as much as an engineering one. Timeline to resolve: 1–2 weeks if proxy route is taken.

PoC Success Criteria

Run of at least 10 million URLs:

Criterion	Pass Threshold
Crawl throughput	$\geq 1,000$ URLs/sec sustained for 1 hour
Success rate	$\geq 90\%$ reach <code>success</code> or <code>blocked</code>
Data completeness	$< 5\%$ of successful crawls have null title
Topic extraction	$\geq 80\%$ of successful crawls produce ≥ 1 topic
End-to-end latency P95	≤ 30 minutes
Cost	Within 120% of Part 2 estimate

Release Plan

Alpha — Internal (1M URLs) Controlled test set covering all page types. Manual review of metadata and topic quality. Gate: all PoC criteria pass.

Beta — Limited Production (100M URLs) Production domain lists. On-call active. Tune httpx vs Playwright routing from failure patterns. Gate: throughput $\geq 3,000$ URLs/sec, error rate $< 5\%$, no data loss.

General Availability Full 5,000 URLs/sec. All Part 2 SLAs enforced. Runbooks rehearsed, DR tested. Gate: two consecutive clean beta runs with no P1 incidents.